

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Выполнила:

Мищенко Арина

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках лабораторной работы №2 необходимо подключить веб-приложение платформы для онлайн-курсов и обучения, разработанное в LP1, к внешнему API средствами Fetch API. В качестве API используется моковый сервер на основе JSON Server, имитирующий работу реального бэкенда.

Требовалось реализовать следующее:

1. Создать файл базы данных db.json с коллекциями пользователей, курсов, записей на курсы, заданий, выполненных заданий и обсуждений.
2. Подключить авторизацию: при входе проверять учётные данные через API, при регистрации — создавать нового пользователя через POST-запрос.
3. Загружать каталог курсов из API (только опубликованные), фильтровать на стороне клиента.
4. В личном кабинете студента получать список записей и данные курсов через API, отображать прогресс и сертификаты.
5. На странице курса загружать данные по id из URL, реализовать запись на курс, систему сдачи и проверки заданий с несколькими статусами.
6. Реализовать кабинет преподавателя: создание курсов, публикация черновиков, добавление заданий, просмотр и проверка работ студентов с автоматическим обновлением прогресса.
7. Реализовать раздел обсуждений: загрузка и создание тем через API.
8. Обеспечить корректную обработку несоответствия типов данных между клиентом и сервером.

Ход работы

1. Настройка JSON Server

JSON Server — инструмент, который поднимает полноценный REST API из одного JSON-файла без написания серверного кода. Установка и запуск:

```
npm install -g json-server
json-server --watch db.json --port 3000
```

После запуска сервер доступен по адресу `http://localhost:3000` и автоматически создаёт REST-эндпоинты для каждой коллекции. JSON Server поддерживает все основные HTTP-методы, фильтрацию через query-параметры, а также автоматически генерирует уникальные id при POST-запросах. Идентификаторы генерируются как строки вида «YLW17EIL9HI» (Base62) — это важная особенность, которая повлияла на реализацию фильтрации (подробнее в п. 3.1).

2. Структура db.json

Файл db.json содержит шесть коллекций:

- users – пользователи: id, firstName, lastName, email, password, role ('student' / 'teacher'). Два тестовых аккаунта: `ivan@example.com` и `teacher@example.com`, пароль 123456.

- courses – курсы: id, title, description, category, level, price, teacher, lessons, hours, rating, reviews, color, icon, teacherId, status ('published' / 'draft').
- enrollments – записи студентов на курсы: id, userId, courseId, progress, status ('active' / 'completed').
- tasks – задания курсов: id, courseId, lesson, number, title, description.
- completedTasks – сданные задания: id, userId, courseId, taskId, answer, reviewed, result ('accepted' / 'rejected' / null), createdAt.
- discussions – темы обсуждений: id, courseId, userId, authorName, title, message, createdAt.

Поля password хранятся в открытом виде, поскольку это учебная работа с имитацией API без настоящей системы безопасности. Поле teacherId в курсах позволяет фильтровать курсы по преподавателю на клиенте. Поле status разделяет опубликованные курсы (видны в каталоге) и черновики (видны только преподавателю).

3. Авторизация (login.html)

При отправке формы функция `handleLogin()` получает всех пользователей с совпадающим email и проверяет пароль и роль на клиенте:

```
const response = await fetch(`${API}/users?email=${email}`);
const users = await response.json();
const user = users.find(u => u.password === password && u.role === currentRole);
```

Такой подход выбран осознанно: изначально запрос отправлялся с тремя параметрами через URL (email, password, role), однако `encodeURIComponent()` кодирует символ `@`, а JSON Server не выполняет обратное декодирование при матчинге строк. В результате пользователь не находился. Решение – фильтровать по email без кодирования (`@` допустим в query string), а пароль и роль проверять через `Array.find()` на клиенте.

На время запроса кнопка блокируется через `submitBtn.disabled = true`. Вся логика обернута в `try/catch/finally` — `finally` гарантирует разблокировку кнопки даже при ошибке. Данные сохраняются в `sessionStorage` (role, userId, firstName, lastName) и используются на всех последующих страницах.

3.1 Проблема несоответствия типов и её решение

В процессе разработки обнаружена нетривиальная проблема: JSON Server v1 использует строгое сравнение типов при фильтрации через query-параметры. URL-параметры всегда передаются как строки, тогда как в `db.json` поля id могут быть числами или строками в зависимости от того, кто создал запись (предустановленные данные имели числовые id, а созданные через POST — строковые). Это приводило к тому, что записи о прохождении курсов не находились при фильтрации.

Решение — отказаться от серверной фильтрации по `userId/courseId` и делать её на клиенте с оператором строгого равенства после приведения обоих значений к строке:

```
const enrollments = allEnrollments.filter(
  e => String(e.userId) === String(userId)
);
```

Оператор `String()` гарантирует совпадение независимо от того, хранится ли `userId` как число 1, строка "1" или Base62-идентификатор "YLW17EIL9HI". Аналогичный подход применён во всех местах сравнения `userId` и `courseId` в проекте.

4. Регистрация (register.html)

Функция `handleRegister()` выполняет последовательно два запроса. Сначала проверяется уникальность email:

```
const checkRes = await fetch(`${API}/users?email=${email}`);
const existing = await checkRes.json();
if (existing.length > 0) { /* email занят */ }
```

При успехе отправляется POST-запрос для создания пользователя. JSON Server автоматически присваивает уникальный id и возвращает созданный объект. Данные сразу сохраняются в `sessionStorage` для авторизации без повторного входа:

```
const createRes = await fetch(`${API}/users`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(newUser)
});
const createdUser = await createRes.json(); // содержит сгенерированный id
```

5. Каталог курсов (search.html)

При загрузке страницы функция `loadCourses()` получает все курсы и фильтрует только опубликованные:

```
const allData = await response.json();
allCourses = allData.filter(c => c.status === 'published' || !c.status);
```

Это позволяет преподавателям работать с черновиками, не показывая их студентам. Карточки курсов генерируются динамически функцией `createCourseCard(course)` через шаблонные строки и вставляются через `.map().join('')`. Ссылка «Подробнее» ведёт на `course.html?id=N`. Фильтрация по предмету, уровню и цене выполняется на клиенте по уже загруженному массиву без повторных запросов к серверу.

6. Личный кабинет студента (dashboard-user.html)

Функция `loadEnrollments()` загружает все записи и фильтрует по `userId` на клиенте. Затем для каждой записи параллельно запрашиваются данные курса через `Promise.all()`:

```
const coursePromises = enrollments.map(e =>
  fetch(`${API}/courses/${e.courseId}`).then(r => r.json())
);
```

```
const courses = await Promise.all(coursePromises);
```

`Promise.all()` — ключевая конструкция: она запускает все `fetch`-запросы одновременно и ожидает завершения всех. Если бы запросы выполнялись последовательно в цикле (с `await` внутри `forEach`), общее время ожидания было бы суммой времён всех запросов. С `Promise.all()` оно равно времени самого медленного запроса.

Прогресс в карточках курсов отражает долю принятых преподавателем заданий. Сертификаты отображаются только при `enrollment.status === 'completed'`, что устанавливается автоматически при принятии всех заданий. Профиль обновляется через PATCH-запрос, изменяющий только имя и фамилию.

7. Страница курса (course.html)

Id курса извлекается из URL через `URLSearchParams`:

```
const params = new URLSearchParams(window.location.search);  
const courseId = params.get('id') || '1';
```

Все данные загружаются параллельно: курс, статус записи, задания, выполненные задания. Состояния заданий хранятся в объекте-карте `taskStateMap` (ключ — `taskId`, значение — объект с полями `ctId`, `reviewed`, `result`, `answer`). Это позволяет за $O(1)$ получить состояние любого задания:

```
taskStateMap = {};  
allDone.filter(d => String(d.userId) === String(userId) &&  
    String(d.courseId) === String(courseId))  
    .forEach(d => {  
        taskStateMap[d.taskId] = { ctId: d.id, reviewed: d.reviewed,  
                                   result: d.result, answer: d.answer  
    };  
    });
```

Задание может находиться в одном из четырёх состояний, отображаемых визуально:

- Не сдано — кнопка «Открыть»
- На проверке — (`reviewed: false`), бейдж «На проверке», форма скрыта
- Принято — иконка зелёная (`result: 'accepted'`), кнопка «Просмотр»
- На доработку — иконка красная (`result: 'rejected'`), кнопка «Переотправить»

При пересдаче старая запись удаляется через `DELETE`, затем создаётся новая через `POST` — это гарантирует, что в базе всегда не более одной записи на задание от студента. Прогресс-бар рассчитывается только по принятым заданиям:

```
const acceptedCount = Object.values(taskStateMap)  
    .filter(s => s.result === 'accepted').length;  
const pct = Math.round((acceptedCount / total) * 100);
```

Обновление прогресса в enrollment происходит не при сдаче задания студентом, а при принятии преподавателем. Это разделение ответственности обеспечивает корректность данных: прогресс отражает реальные результаты, а не просто факт отправки.

8. Кабинет преподавателя (dashboard-teacher.html)

Весь кабинет построен на fetch-запросах. При загрузке из sessionStorage считывается userId преподавателя, затем загружаются все курсы и фильтруются по teacherId на клиенте.

Создание курса выполняется через POST в коллекцию /courses. Цвет и иконка подбираются автоматически по категории через объект-словарь colorMap. Статус задаётся кнопкой: «Сохранить черновик» передаёт status: 'draft', «Опубликовать» — status: 'published'. Публикация черновика делается через PATCH:

```
await fetch(`${API}/courses/${courseId}`, {
  method: 'PATCH',
  body: JSON.stringify({ status: 'published' })
});
```

Добавление заданий в курс — POST в /tasks с полями courseId, lesson, number, title, description. После сохранения список заданий обновляется без закрытия модалки.

Раздел «Задания студентов» загружает три коллекции параллельно через Promise.all(): completedTasks, tasks, users. Затем непроверенные задания по курсам преподавателя обогащаются именем студента и названием задания:

```
const [ctRes, tasksRes, usersRes] = await Promise.all([
  fetch(`${API}/completedTasks`),
  fetch(`${API}/tasks`),
  fetch(`${API}/users`)
]);
```

При принятии задания функция setGradeResult() выполняет каскадное обновление: сначала PATCH completedTask (reviewed: true, result: 'accepted'), затем пересчитывает количество принятых заданий студента по этому курсу и делает PATCH enrollment с новым прогрессом. Если прогресс достигает 100%, статус меняется на 'completed' и у студента появляется сертификат:

```
const acceptedCount = allCT.filter(ct =>
  String(ct.userId) === String(studentId) &&
  String(ct.courseId) === String(courseId) &&
  ct.result === 'accepted'
).length;
const pct = Math.round((acceptedCount / courseTasks.length) * 100);
await fetch(`${API}/enrollments/${enrollment.id}`, {
  method: 'PATCH',
  body: JSON.stringify({ progress: pct, status: pct === 100 ?
    'completed' : 'active' })
});
```

9. Обсуждения (course.html)

Темы обсуждений хранятся в коллекции `/discussions`. При открытии вкладки загружаются все обсуждения и фильтруются по `courseId` на клиенте. Создание новой темы – POST с полями `courseId`, `userId`, `authorName`, `title`, `message`, `createdAt`:

```
createdAt: new Date().toISOString().slice(0, 10)
```

После успешного POST модалька закрывается программно через `bootstrap.Modal.getInstance(element).hide()` (не через `data-bs-dismiss`), что позволяет добавить проверку полей перед закрытием. Затем вызывается `loadDiscussions()` для обновления списка без перезагрузки страницы.

Вывод

В ходе лабораторной работы веб-приложение платформы для онлайн-курсов было полностью подключено к моковому API на основе JSON Server. Все операции с данными реализованы через HTTP-запросы с использованием Fetch API.

Применены следующие типы запросов: GET для получения коллекций с клиентской фильтрацией, POST для создания пользователей, записей на курсы, заданий, сданных работ и тем обсуждений, PATCH для частичного обновления (профиль, статус курса, результат задания, прогресс enrollment), DELETE для удаления передаваемых заданий.

В процессе разработки решены нетривиальные задачи: устранена проблема несоответствия типов данных между клиентом и JSON Server v1 через `String()`-приведение, исправлена ошибка кодирования символов в URL через переход на клиентскую проверку пароля, реализована каскадная логика обновления прогресса при проверке заданий преподавателем. Для параллельных запросов использован `Promise.all()`, для хранения состояний заданий — объект-карта `taskStateMap`, для разбора URL — встроенный `URLSearchParams`. Все асинхронные операции обернуты в `async/await` с обработкой ошибок через `try/catch/finally`.